Fase 1 — Diagnóstico do cérebro

K.E.R.N.E.L 0.5.1 · 15/10/2026 · branch `estabilizacao-0.5`

Pergunta da fase: por que o modelo atual não gera texto coerente, e como medir progresso de um jeito confiável.

A resposta curta:

1. **O pipeline está certo.** Cada peça foi conferida contra uma referência independente. O mesmo tamanho de modelo, na mesma placa, decorou um texto e o repetiu sozinho, 100% igual. Nenhum erro de treino ou de inferência explica o texto sem sentido.
2. **O modelo é pequeno demais para o que se pede dele.** Com 3,5 milhões de parâmetros, ele prevê o próximo token pior que uma simples tabela de bigramas quando só vê um token de contexto. Também quase não decora o que leu: a perda é a mesma no texto já lido e no nunca lido. Ele usa o contexto longo, mas ganha pouco com isso. Sai português com gramática, sem assunto.
3. **O procedimento de treino desperdiça dados e tempo,** e isso custa em torno de 0,1 a 0,15 nat:
   - o sorteio dos lotes repete muito;
   - cada sessão nova reaquece a taxa de aprendizado e desfaz parte do que a anterior fez;
   - a validação antiga era enviesada e barulhenta.

   Isso é real e tem conserto, mas não explica a falta de coerência.
4. **Os "22%" eram ainda menos, e as interrupções custaram quase nada.** Até o passo 20.000, o cérebro tinha lido de fato 13% do corpus. Hoje leu cerca de 30%. Ler mais ajuda, mas devagar: dobrar o texto lido melhorou a perda em 0,09 nat. As interrupções do treino, juntas, perderam menos de 300 passos.

A linha de base, no conjunto de teste fixo (2 milhões de tokens nunca lidos):

Modelo	Perda (nats/token)	Perplexidade	Bits por byte
O cérebro do serviço (passo 46.000)	**3,094**	22,1	1,379
Trigrama contado (Kneser-Ney)	3,719	41,2	—
Unigrama	6,882	974	—

Medida reprodutível: o mesmo modelo, medido três vezes, deu os mesmos números em todas as casas decimais.

---

1. Como foi medido

Ferramenta	O que faz	Comando
Conjuntos fixos (`kernel/cerebro/conjuntos.py`)	Teste, validação e uma amostra do treino. Teste e validação vêm de documentos que o cérebro nunca leu e saem do treino; impressão SHA-256 de cada arquivo	`python -m kernel cerebro conjuntos --sessoes ...`
Sorteio refeito (`kernel/cerebro/cobertura.py`)	Refaz, chamada por chamada, o sorteio dos lotes de cada sessão e diz exatamente o que o cérebro já leu	(usado pelos conjuntos)

Ferramenta	O que faz	Comando
Checagens do pipeline (`kernel/cerebro/verificar.py`)	10 checagens contra referências independentes, inclusive o teste de memorizar	`python -m kernel cerebro verificar`
Protocolo de avaliação (`kernel/cerebro/protocolo.py`)	Perda em cada token dos conjuntos fixos, por posição na janela, n-gramas, textos com começos e sementes fixos, velocidade e memória	`python -m kernel cerebro protocolo`
Experimentos (`docs/fase1/experimentos.py`)	A partir de um mesmo checkpoint: continuar com a taxa baixa, continuar reaquecendo, e repetir para medir a reprodutibilidade	`python docs/fase1/experimentos.py ...`

Os modelos medidos foram congelados antes, para o treino em andamento não trocá-los no meio das medidas:

Modelo	O que é	Texto distinto lido
Passo 20.000	O modelo auditado na 0.5.0	13,1%
Passo 46.000	O modelo do serviço hoje	27,0%
Passo 49.039	O fim da sessão de 1 h de 15/10	28,6%
Passo 50.000	No meio de uma sessão reaquecida	~29%

Uso da placa nesta fase: cerca de 80 minutos, dentro das 2 horas combinadas.

2. O pipeline está certo

`python -m kernel cerebro verificar --memorizar mini --dispositivo directml` (resultado completo em `dados/fase1/verificar-mini-directml.txt`):

Checagem	Contra o quê	Resultado
Tokenizador	Ida e volta texto → tokens → texto; e os tokens gravados no treino contra os que ele produz hoje	Sem perder um byte; os tokens gravados são exatamente os de hoje; 3,17 caracteres por token
Alvos	Um arquivo em que cada token vale a sua posição	O alvo é sempre o token seguinte, inclusive depois de trocar de pedaço (50 de 50 lotes)
Máscara causal	Trocar um token do futuro e ver se o passado muda	Mudança no passado: exatamente 0
Normalização, GELU, atenção, perda	As funções prontas do PyTorch (`layer_norm`, `gelu`, `scaled_dot_product_attention`, `cross_entropy`), em float64	Diferenças de 10^{-12} ou menores
Inicialização	A teoria: perda inicial ≈ ln(vocabulário) + $s^2/2$, para notas iniciais de desvio s	8,369 contra 8,356 esperados (ln 4096 = 8,318); a via residual não explode
Gradientes	Um GPT de referência feito só com funções do PyTorch, e diferenças finitas	$3{,}9 \cdot 10^{-14}$ contra a referência; $1{,}2 \cdot 10^{-9}$ contra as diferenças finitas
Otimizador	`torch.optim.AdamW` com o mesmo decaimento só nas matrizes, e `clip_grad_norm_`	$2 \cdot 10^{-10}$ depois de 6 passos; corte idêntico

Checagem	Contra o quê	Resultado
Checkpoint	Treinar, salvar, restaurar e continuar, contra treinar sem parar	Pesos idênticos (diferença 0) dois passos depois de restaurar; impressão digital confere
Inferência do serviço	As notas do NumPy (o que o serviço usa) contra as do PyTorch, nos pesos reais	$4{,}4 \cdot 10^{-5}$, o mesmo token mais provável em 100% das posições; o atalho de um token por vez bate com a conta inteira
Memorizar	Treinar só num texto de 257 tokens e pedir que ele o continue sozinho, pela inferência do serviço	**Mini (3,5 mi) na RX 570: perda 0,0000 em 300 passos, repetiu 100% dos 248 tokens.** Nano na CPU: perda 0,0012, 100%

Conclusão: treino e inferência estão validados. O texto sem sentido não vem de erro no pipeline.

3. Os dados

3.1 A validação antiga não representava o corpus

Até a 0.5.1, a validação era o último 1% do corpus (2 milhões de tokens), ou seja, o fim do dump da Wikipédia:

- **Artigos recentes, em blocos de um assunto só.** Batalhas, depois álgebra, depois economia. Com mediana de 955 tokens por artigo, contra 583 no treino.
- **O corte caiu no meio de um artigo.** O primeiro documento da validação é a segunda metade de um artigo que está no treino.
- **A medida durante o treino era barulhenta.** Cada medida sorteava 20 lotes novos, e entre duas medidas vizinhas a diferença oscilava ±0,035. O "melhor 3,160" do passo 20.000 era o ponto mais baixo de uma série barulhenta. Medido de verdade, o modelo do passo 20.000 fica em 3,195 nessa mesma validação antiga.

3.2 Os conjuntos fixos novos

Regra (no `manifesto.json`):

- só documentos que o cérebro nunca leu;
- tirados de regiões que nenhuma sessão chegou a carregar (pedaços de 8 milhões de tokens);
- sem cópia exata;
- de 32 a 16.384 tokens;
- sorteio com semente fixa.

Teste e validação saem do treino, para continuarem nunca lidos. O `treino.bin` antigo ficou guardado como `treino-original.bin`.

Por que só de regiões nunca carregadas: artigos longos têm mais chance de terem sido tocados por alguma janela. Sortear entre os "não tocados" de qualquer lugar puxaria o teste para os artigos curtos.

Conjunto	Documentos	Tokens	Mediana por documento	Trechos de 13 tokens que aparecem iguais no treino
Teste	1.810	2.000.197	579	6,1%
Validação	902	1.001.244	564	6,7%

Conjunto	Documentos	Tokens	Mediana por documento	Trechos de 13 tokens que aparecem iguais no treino
Amostra do treino (já lidos)	821	1.000.295	609	—
Treino que sobrou	644.025	768.432.182	583	—

As medianas batem com a do treino. Os 6% de trechos repetidos são frases-modelo da Wikipédia, como "é uma comuna francesa na região administrativa de...", e não artigos copiados: nenhum documento tem cópia exata, porque o título faz parte de cada um. O treino passou a escolher o melhor modelo pela validação nova, sempre nas mesmas 320 janelas.

3.3 O que o cérebro de fato leu

O sorteio de cada sessão foi refeito a partir das sementes conhecidas (`1337 + passo inicial`). Um teste confere a reconstrução contra a classe `Lotes` de verdade, janela por janela, inclusive nas trocas de pedaço.

As sessões foram reconstruídas do `treino.csv`, do `treino.log` e do registro:

Sessão	Passos
1	0 → 1.386
2	1.386 → 20.000
3	20.000 → 20.573
4	20.573 → 46.050
5	46.000 → 49.039
6	49.039 → ~50.200

Até o passo	Tokens processados	Tokens distintos lidos	Documentos tocados
20.000	164 mi (21,2%)	101 mi (**13,1%**)	19,5%
46.000	377 mi (48,8%)	208 mi (27,0%)	37,3%
~50.300	412 mi (53,5%)	228 mi (29,6%)	40,6%

O sorteio repete muito. Do que passou pelo treino, 38% já tinha sido lido até o passo 20.000, e 45% até hoje. Ele carrega um pedaço de 8 milhões de tokens e sorteia janelas dentro dele. Uma passada assim cobre só ~63% do pedaço e repete o resto. Os pedaços também se sobrepõem.

4. A linha de base

O protocolo (`docs/fase1/linha-de-base/`, um relatório por modelo):

Modelo	Teste	Validação	Amostra já lida	Validação antiga	Bits por byte (teste)
Passo 20.000	3,185	3,209	3,182	3,195	1,420
Passo 46.000 (serviço)	**3,094**	**3,116**	3,087	3,037	**1,379**
Passo 49.039	3,110	3,133	3,103	3,049	1,386
Passo 50.000 (meio do reaquecimento)	3,241	3,258	3,233	3,251	1,445

As linhas de base de n-gramas foram contadas em 100 milhões de tokens do treino e medidas nos mesmos conjuntos:

	Unigrama	Bigrama	Trigrama	GPT (passo 46.000)
Perda no teste	6,882	4,603	3,719	3,094

A perda por posição na janela mostra quanto o contexto ajuda (passo 46.000, no teste):

Posições	1	2–4	5–16	17–64	65–128	129–256
Perda	5,146	3,967	3,455	3,199	3,069	2,997

Três leituras:

- **Com pouco contexto, o GPT perde para tabelas de contagem.** Com um token só de contexto (posição 1), ele faz 5,15, contra 4,60 do bigrama contado. Com dois ou três tokens, faz 3,97, contra 3,72 do trigrama. 3,5 milhões de parâmetros não guardam nem as estatísticas locais que uma tabela guarda.
- **Ele usa o contexto longo, mas ganha pouco.** Passar de 64 para 256 tokens de contexto rende só 0,2 nat. É assim que ele passa o trigrama no total, mas a melhora é pequena.
- **Ele não decora.** A perda nos documentos que ele já leu (3,087) é quase igual à dos nunca lidos (3,094). Um modelo com sobra de capacidade ficaria bem melhor no que já viu. Este não consegue.

O conjunto de teste é a régua daqui para a frente: 3,094 nats/token, ou 1,379 bits por byte. Os bits por byte não dependem do tokenizador: servem para comparar um cérebro futuro com outro vocabulário.

5. Os "22%" e o treino interrompido

5.1 Ler mais ajuda, mas devagar

	Passo 20.000	Passo 46.000	Diferença
Texto distinto lido	13,1%	27,0%	×2,1
Perda no teste	3,185	3,094	−0,091

Com o mesmo tamanho de modelo, ler o corpus inteiro (3,7 vezes o texto distinto de hoje) deve render mais um pouco, talvez de 0,1 a 0,2 nat. Isso não transforma frases sem sentido em texto coerente. Os dados não lidos são parte da explicação, não a explicação.

5.2 As interrupções custaram quase nada

Quando	O que aconteceu	Perda
13/10 23:44	O Windows fechou o treino junto com o app do Claude, que se atualizou (ver a investigação do A3)	Nenhuma: salvou o checkpoint
15/10 05:12	O PC reiniciou sem desligar direito (Kernel-Power 41: queda de energia ou travamento do sistema)	50 passos (voltou ao checkpoint do passo 46.000)
15/10 20:42	O serviço foi desligado e o treino morreu junto, antes de salvar	200 passos

O caso das 20:42 começou comigo: editei arquivos selados com o serviço ligado. O alarme da Fase 0 ligou o modo seguro e avisou, e o serviço foi desligado.

Isso revelou um defeito real: ao desligar, o serviço mandava o treino parar, mas saía antes de o treino conseguir salvar, e o processo do treino morria junto. **Corrigido:** o serviço agora espera até 60 s o treino salvar.

5.3 O custo de verdade: reaquecer a taxa a cada sessão

Cada sessão nova sobe a taxa de aprendizado de novo, até 3e-4, e desce em cosseno até o fim. No histórico, toda sessão começa com a validação pior que a anterior terminou: 3,03 → 3,24. A sessão de 1 h de 15/10 terminou onde começou (3,033 → 3,035), e o protocolo confirma que o passo 49.039 ficou um pouco pior que o 46.000.

O experimento isolou a causa. Mesmo checkpoint (passo 49.039), mesmos dados, mesmos 1.100 passos (~9 milhões de tokens); só a taxa muda:

Passo	Taxa baixa (3e-5)	Reaquecendo (até 3e-4)
0	3,161	3,161
200	3,155	3,247
300	3,154	**3,271**
600	3,153	3,251
1.100	**3,144**	3,184
Validação inteira no fim	**3,113**	3,150

Reaquecer custa ~0,1 nat no começo e precisa de mais de 1.100 passos (~22 min) só para voltar. Sessões curtas saem no prejuízo.

5.4 O corte de gradiente age sempre

A norma do gradiente fica em 1,79 ± 0,05, e o limite de corte é 1,0. Ou seja, 100% dos passos são cortados. Com o Adam isso quase não muda o passo, porque ele normaliza a escala de cada peso. Mas o corte deixa de servir de proteção contra picos. Não houve picos nas medidas: o máximo foi 2,2.

6. A geração

O protocolo gera 60 tokens a partir de 12 começos fixos, de três jeitos:

- **guloso:** sempre o mais provável;
- **sorteio:** temperatura 0,8, os 40 mais prováveis, 3 sementes;
- **como o serviço faz:** com top-p 0,92 e penalidade de repetição.

Médias, passo 46.000:

Modo	Trios de tokens repetidos	Pares de palavras distintos	Palavras que existem	Repete alguma palavra do começo
Guloso	54,8%	41,8%	99,8%	75%
Sorteio	5,7%	92,2%	94,3%	44%
Como o serviço	0,0%	98,4%	94,0%	42%

Exemplos (todos estão em `linha-de-base/passo-46000/relatorio.md`):

- **guloso** · *A fotossíntese é o processo* de transmissão de dados de dados de dados de dados de dados…
- **serviço** · *O Brasil é* um jogo de golfe com menos de 15.000 libras em seu tempo, com um recorde de 24 pontos de 33…
- **serviço** · *A Segunda Guerra Mundial*. Foi detido em 1938, mas a guerra foi suspensa e o tratado não foi bem recebido pela Alemanha…

Avaliação à mão dos 12 textos do modo do serviço, de 0 a 2 em cada critério:

Critério	Pontos	Porcentagem
Português (gramática e palavras)	22 de 24	92%
Fica no assunto do começo	6 de 24	25%
Coerência (as frases fazem sentido juntas e com o mundo)	2 de 24	8%

Quem avaliou foi eu (Claude), sozinho: é subjetivo. Os textos estão no relatório para você avaliar também.

O retrato bate com a seção 4: frases bem formadas, palavras reais, nenhuma ideia que dure mais que uma oração. No modo guloso, entra em laço.

7. Recursos e estabilidade

Medida	Treino (mini, lote 16 × 2)	Avaliação (protocolo)
Velocidade	6.900 a 7.100 tokens/s	25.700 tokens/s; geração em NumPy, um token por vez: 247 tokens/s
VRAM	3,18 GB do processo (3,86 GB dos 4 GB da placa em uso)	3,19 GB de pico (o DirectML reserva quase tudo)
RAM	415 MB em uso; 3,6 GB reservados	400 MB de pico
Reprodutibilidade	Duas execuções iguais de 300 passos: diferença máxima de $4{,}8 \cdot 10^{-7}$ na perda	Três rodadas no mesmo modelo: relatório idêntico em todas as casas decimais
Estabilidade	Norma do gradiente 1,79 ± 0,05, sem picos (máximo 2,2); nenhuma perda infinita	—

Não cabe outro trabalho na placa enquanto o treino roda.

8. Diagnóstico

O texto não tem sentido porque **o modelo não tem capacidade para guardar o que precisaria**. Evidências:

1. Perde para um bigrama e um trigrama contados quando o contexto é curto (seção 4).
2. Não decora o que lê: a perda é igual no texto lido e no nunca lido (seção 4).
3. Mais texto ajuda devagar: o dobro de texto lido rendeu 0,09 nat (seção 5.1).
4. Usa o contexto longo, mas ganha pouco com ele: 0,2 nat de 64 para 256 tokens (seção 4).
5. A geração confirma: gramática, sem assunto (seção 6).

Pioram a perda, mas não explicam a falta de coerência:

- o sorteio que repete (de 38% a 45% de repetição);
- o reaquecimento a cada sessão (~0,1 nat temporário, e sessões curtas sem ganho);
- a validação que enganava a escolha do "melhor" modelo.

Descartados como causa:

Suspeito	Por que foi descartado
Erros no pipeline	Nenhum encontrado; o modelo decora um texto pequeno com perfeição (seção 2)
Tokenizador	Sem perda e com boa compressão: 3,17 caracteres por token
Interrupções	Menos de 300 passos perdidos no total

O que isso significa: com 3,5 milhões de parâmetros, dá para ganhar mais um pouco ajustando o treino e lendo mais texto, talvez de 0,1 a 0,2 nat. Coerência pede um modelo maior. Esse é o assunto da fase 4 do plano, que não foi iniciada e depende da sua permissão. O protocolo e os conjuntos fixos já estão prontos para medir um cérebro maior com a mesma régua.

9. O que mudou no código

- **Conjuntos fixos.** `kernel/cerebro/cobertura.py` e `conjuntos.py`. O `treino.bin` agora não contém o teste nem a validação; o original ficou em `dados/cerebro/treino-original.bin`, e o `manifesto.json` diz de onde veio cada documento.
- **Validação do treino.** Usa a validação nova, sempre nas mesmas 320 janelas. Se a validação mudar, a "melhor perda" é refeita medindo o modelo exportado, para um modelo pior nunca substituir o atual por engano. A norma do gradiente aparece no progresso.
- **Comandos novos.** `cerebro verificar`, `cerebro conjuntos` e `cerebro protocolo` (`kernel/cerebro/verificar.py`, `ngramas.py`, `protocolo.py` e `pesos.py`).
- **Desligar.** O serviço espera o treino salvar antes de sair.
- **Corrida no treino pendente.** O arquivo do "treino pendente" é gravado antes de o treino começar. Antes, um treino muito rápido o apagava antes de ele existir, e um teste falhava às vezes.
- **Testes.** 360 (eram 353): nenhuma falha, 1 pulado (o de sempre). Resultado em `testes-fase1.xml`.

10. Recomendações

Não foram executadas nesta fase. A decisão é sua.

1. **Continuar sessões sem reaquecer.** Retomar com a taxa onde a sessão anterior parou, ou com uma taxa baixa e constante. O experimento da seção 5.3 mostra o ganho.
2. **Sorteio sem repetição.** Percorrer o corpus em ordem embaralhada por documento, sem sortear janelas com reposição. Cada passo passa a ler texto novo, em vez de 38% a 45% de repetição.
3. **Corte de gradiente acima da norma típica,** por exemplo 3,0, para voltar a ser uma proteção. Hoje ele corta sempre.
4. **Para coerência, um modelo maior.** É a fase 4 do plano, e só com a sua permissão. Antes disso, os itens 1 e 2 deixam o mini no melhor que ele pode dar, e o protocolo mede o ganho de cada mudança separada.

11. Portão de saída

Exigência	Situação
Treinamento validado	Sim: camadas, gradientes, otimizador, corte, checkpoint e retomada conferidos contra referências; treino reprodutível na placa
Inferência validada	Sim: a inferência do serviço bate com o PyTorch nos pesos reais
O modelo aprende um conjunto pequeno de propósito	Sim: o mini na RX 570 decorou 257 tokens e os repetiu sozinho (100%)
Linha de base estável	Sim: conjuntos fixos com impressão digital; protocolo idêntico em três rodadas; teste = 3,094 nats/token no modelo do serviço
"Não avançar só porque a perda caiu"	Respeitado: a perda caiu de 3,185 para 3,094 do passo 20.000 ao 46.000, e a coerência continua em 8% na avaliação à mão. A coerência é medida à parte da perda

12. Como reproduzir

```
.venv\Scripts\python.exe -m kernel cerebro verificar --memorizar mini --dispositivo directml
.venv\Scripts\python.exe -m kernel cerebro protocolo --modelo dados\cerebro\modelo.npz
.venv\Scripts\python.exe docs\fase1\experimentos.py --checkpoint dados\fase1\checkpoint-
49039.pt --saida dados\fase1\experimentos
```

Os conjuntos fixos já existem: `dados/cerebro/avaliacao/`, conferidos pelo manifesto a cada medida. Para montá-los de novo num corpus novo:

```
.venv\Scripts\python.exe -m kernel cerebro conjuntos --sessoes "0-1386,1386-20000,..."
```

Limitações

- **Uma pessoa só avaliou a coerência:** eu, Claude.
- **Teste só de Wikipédia.** O teste é tirado do mesmo corpus. Mede o que o cérebro aprendeu da Wikipédia, não de outros textos.
- **A reconstrução do que foi lido** supõe que a lista de sessões está completa. Ela foi cruzada com o `treino.csv`, o `treino.log` e o registro. O código do sorteio não mudou desde antes do primeiro treino. E o fim de cada sessão foi contado do jeito conservador: mais passos contados como lidos só deixam mais documentos fora do teste.
- **Faixa de ganho estimada.** A estimativa de 0,1 a 0,2 nat para o mini vem de duas medidas: dobrar o texto rendeu 0,09, e a taxa baixa melhorou ~0,02 em 9 milhões de tokens. Não é uma lei medida.